Übungsblatt 3: Fortgeschrittene Interrupts und Timer

Mikrocomputertechnik 3 (MCT3)

Musterlösung

Zielsetzung

Sie vertiefen Vorlesung 3: Direct vs. Vectored Mode (`mtvec`), CLINT/PLIC, Nested Interrupts/`mepc`-Clobbering sowie die Programmierung eines periodischen System-Ticks über `mtime`/`mtimecmp`.

Bearbeitungszeit: ca. 45–60 Min. | **Orientierungspunkte:** Aufgabe 1: 12 P, Aufgabe 2: 10 P (Gesamt: 22 P)

Teil 1: Theorie — Interrupt-Architektur auf SoCs

Aufgabe 1.1: Direct vs. Vectored Mode

Ein RISC-V-SoC empfängt Interrupts von verschiedenen Quellen. In `mtvec` steht die Basisadresse (`BASE`); das `MODE`-Feld in den untersten Bits steuert das Einsprungverhalten.

1. Erklären Sie den Ablauf im **Direct Mode** ($MODE = 0$). Warum stößt dieser Modus bei vielen Peripheriequellen an Grenzen (Stichwort: Suchlatenz)?
2. Erklären Sie den Ablauf im **Vectored Mode** ($MODE = 1$). Wie berechnet die Hardware die Einsprungadresse für Exception-Code 7 (Machine Timer Interrupt)?

Musterlösung Aufgabe 1.1

1. Im **Direct Mode** springt die Hardware bei jedem Trap/Interrupt an dieselbe Adresse `BASE`. Die Software liest `mcause` und entscheidet per Verzweigungskette, welche Quelle vorliegt. Bei vielen Quellen wächst die Such-/Entscheidungszeit (im schlechten Fall linear mit der Zahl der Fälle) und die Latenz schwankt — ungünstig für harte Echtzeit.
2. Im **Vectored Mode** (für Interrupts) bildet die Hardware einen Eintrag in einer Sprungtabelle:

   $$\text{PC} = \text{BASE} + 4 \times \text{ExceptionCode}$$

   Für Code 7 (Machine Timer): Offset $7 \times 4 = 28$ Byte, Zieladresse BASE + 28. An dieser Stelle liegt in RISC-V typisch ein **Sprungbefehl** (`j`/`jal`) zur konkreten ISR — keine Adresse wie bei manchen Cortex-M-Tabellen.

Aufgabe 1.2: CLINT vs. PLIC

Interrupt-Verwaltung auf RISC-V-SoCs: Core-Local Interruptor (CLINT) und Platform-Level Interrupt Controller (PLIC).

1. Nennen Sie die Hauptaufgaben des CLINT und erklären Sie, warum er „Core Local“ ist (Vorteil gegenüber dem PLIC).
2. Nennen Sie die Hauptaufgaben des PLIC.
3. Der PLIC nutzt einen Zwei-Stufen-Handshake. Erklären Sie **Claim** und **Complete**. Warum ist das Auslesen der Claim-ID auf Hardware-Ebene atomar?

Musterlösung Aufgabe 1.2

1. **CLINT:** lokale Timer (`mtime`/`mtimecmp`) und Software-Interrupts (u. a. IPIs zwischen Kernen). „Core Local“, weil die Anbindung eng an den jeweiligen Hart/Kern ist — geringe Latenz für kritische OS-Ereignisse (z. B. Scheduler-Tick), ohne hinter vielen externen Quellen im PLIC anstehen zu müssen.
2. **PLIC:** sammelt viele externe Peripherie-IRQs (UART, SPI, GPIO, …), filtert per Enable/Priorität und signalisiert dem Kern typisch einen gebündelten Machine External Interrupt. Software ermittelt die konkrete Quelle über Claim.
3. **Claim:** Lesen der Sieger-ID aus dem Claim/Complete-Register — die Hardware löscht dabei atomar das zugehörige Pending (bzw. markiert die Quelle als in Bearbeitung), sodass z. B. bei Multicore nicht zwei Kerne denselben IRQ bearbeiten. **Complete:** Schreiben der ID zurück signalisiert dem PLIC, dass die ISR fertig ist und die Quelle wieder neue Pending-Zustände melden darf.

Aufgabe 1.3: Nested Interrupts und Clobbering

Beim Trap-Eintritt setzt die Hardware typisch `MIE` in `mstatus` auf 0 — weitere Interrupts sind global gesperrt.

1. Warum ist diese „Taubheit“ bei langen ISRs (z. B. langsame I2C-Sequenz) in Echtzeitsystemen problematisch?
2. Bei Nested Interrupts setzt Software `MIE` in der ISR wieder auf 1. Erklären Sie das Clobbering-Problem bezüglich `mepc`.
3. Was muss Software tun, bevor `MIE` wieder auf 1 gesetzt wird?

Musterlösung Aufgabe 1.3

1. Solange `MIE = 0`, werden höherpriore Ereignisse (z. B. 1-ms-Timer-Tick) verzögert oder verpasst — die Systemzeitbasis und Deadlines leiden.
2. Beim ersten Interrupt speichert die Hardware die Rücksprungadresse (z. B. aus `main`) in `mepc`. Trifft bei wieder freigegebenem `MIE` ein zweiter Interrupt ein, überschreibt die Hardware `mepc` mit der Adresse *innerhalb* der äußeren ISR. Die ursprüngliche Rücksprungadresse geht verloren (**Clobbering**) — nach Nesting droht Absturz oder falscher Rücksprung.
3. Vor dem Re-Enable von `MIE`: `mepc` (und typisch relevante Status-/CSR-Kontexte) auf den Stack retten; nach der inneren Arbeit wiederherstellen. Erst dann ist Nesting sicher.

Teil 2: Praxis — System-Ticks programmieren

Spezifikation (didaktisches Beispiel, Adressen entsprechen dem verbreiteten SiFive-CLINT-Layout):

- CLINT-Oszillator: 10 MHz (10 000 000 Hz)
- `mtime` an `0x0200BFF8` (64 Bit)
- `mtimecmp` an `0x02004000` (64 Bit)

Aufgabe 2.1: Mathematik des Timers

Berechnen Sie die Anzahl der Oszillator-Takte für exakt 1 ms.

Musterlösung Aufgabe 2.1

$$\frac{10\,000\,000 \text{ Takte/s}}{1000} = 10\,000 \text{ Takte}$$

pro Millisekunde.

Aufgabe 2.2: Initialisierung in C

Vervollständigen Sie das Snippet, sodass der erste Alarm in 1 ms ausgelöst wird.

```
#include <stdint.h>

#define CLINT_MTIMECMP 0x02004000u
#define CLINT_MTIME    0x0200BFF8u
#define TICKS_PER_MS   /* TODO: Ergebnis aus 2.1 */

void init_systick(void) {
    /* 1. aktuelle Zeit (mtime) lesen */
    /* TODO */

    /* 2. Intervall addieren, Ziel nach mtimecmp schreiben */
    /* TODO */

    /* 3. Freigabe (Pseudo-Code) */
    /* set_csr(mie, (1u << 7)); */ /* MTIE */
    /* set_csr(mstatus, 8); */     /* MIE */
}
```

Musterlösung Aufgabe 2.2

```
#include <stdint.h>

#define CLINT_MTIMECMP 0x02004000u
#define CLINT_MTIME    0x0200BFF8u
#define TICKS_PER_MS   10000u

void init_systick(void) {
    uint64_t now = *(volatile uint64_t *)CLINT_MTIME;
    *(volatile uint64_t *)CLINT_MTIMECMP = now + TICKS_PER_MS;

    /* set_csr(mie, (1u << 7)); */
    /* set_csr(mstatus, 8); */
}
```

Hinweis: Auf manchen Plattformen müssen 64-Bit-Zugriffe auf `mtime`/`mtimecmp` als zwei 32-Bit-Zugriffe mit Schutz vor Zwischen-Update implementiert werden — hier vereinfacht als ein `uint64_t`-Zugriff. Das Pseudo-`set_csr(mstatus, 8)` steht nur für „Bit MIE setzen“; in echtem Code typisch per Read-Modify-Write/`csrs`, ohne andere `mstatus`-Bits zu löschen.

Aufgabe 2.3: Timer-ISR und Quittieren

Der Timer-IRQ bleibt aktiv, solange `mtime >= mtimecmp`.

1. Warum feuert der Interrupt nach `mret` sofort erneut, wenn die ISR nur eine Variable erhöht und zurückkehrt?
2. Schreiben Sie `void timer_isr(void)`, die `uptime_ms` inkrementiert und den nächsten Alarm in 1 ms plant.

3. Warum sollte `__attribute__((interrupt))` (bzw. ein gleichwertiger ABI-/Prolog) verwendet werden?

Musterlösung Aufgabe 2.3

1. Ohne Erhöhung von `mtimecmp` bleibt `mtime >= mtimecmp` wahr — der IRQ ist weiter pending. Nach `mret` (mit wieder freigegebenen Interrupts) tritt er sofort erneut ein → Interrupt-Sturm / Endlosschleife.

2. Beispiel:

```
volatile uint32_t uptime_ms = 0;

__attribute__((interrupt))
void timer_isr(void) {
    uptime_ms++;

    uint64_t now = *(volatile uint64_t *)CLINT_MTIME;
    *(volatile uint64_t *)CLINT_MTIMECMP = now + TICKS_PER_MS;
}
```

Alternativ robust gegen Drift: `mtimecmp += TICKS_PER_MS` relativ zum *vorherigen* Vergleichswert (statt relativ zu `now`), wenn keine Deadlines verpasst werden sollen.

3. Das Attribut sorgt für ISR-gerechten Prolog/Epilog (Register retten/herstellen) und typisch `mret` statt `ret`. Eine normale C-Funktion darf Caller-Saved-Register zerstören — das würde den unterbrochenen Kontext beschädigen.